If this condition is met, we want to find the best entry to "steal", and usually this means the closest one to the destination node but, as mentioned, other criteria can be used. Anyway, once we find it, we just need to update the source and destination of this transaction, and update them accordingly.

If, instead, the condition above is not met, and there is no sibling of the child violating invariants from which we can borrow an entry, it can mean one of two things:

1. There are no siblings: assuming $m \geq 2$, this can only happen if we are at the root, and it only has 1 child. In this case, there is nothing to do.
2. There are siblings, but all of them have exactly m entries. In this case, since $m \leq M/2$, if we merge the invalid node with any of its siblings, we get a new node with $2*m-1<M$ entries: in other words, a valid node that doesn't violate any invariant.

Listing 10.18 shows how to handle both situations: we check if the second argument is null to understand if we are in the former or latter case, and if we do need to perform a merge, we also clean up parent's node (which is the one node on which the `mergeChildren` method is called).

Listing 10.18 The SsNode:: mergeChildren method

```
function mergeChildren(firstChild,
secondChild)                                    #1
  if secondChild != null
then                                                    #2
```